

Measuring the Local Weight of Coding-Agent Harnesses

A controlled CPU, memory, and task-deployment comparison

mmdmcy
Katteke

19 August 2026

Abstract

Coding-agent harnesses are often compared by model quality or task speed, but a developer running many agents also pays for the local process tree: resident memory, CPU time, child processes, and startup work. This thesis reports a controlled follow-up study of OpenCode, Codex CLI, Cursor Agent CLI, Pi, xAI Grok Build, and Vercel fx. The primary endpoints were peak process-tree resident memory and total user plus system CPU seconds. Twenty fresh version/help trials per harness measured cold startup. Three repeated light and heavyweight coding trials measured deployment behavior, with solo and delegation-enabled arms where the CLI exposed a comparable mechanism.

fx was the lightest cold-start binary at a median 1.47 MB peak RSS. In the matched GPT-5.6 Luna-max solo task subset, Pi had the lowest median peak tree RSS on both light and heavy tasks, Codex used the least CPU on the heavy task, and Cursor finished that task fastest. No harness was best on every metric. These are narrow deployment measurements: task results include model/provider behavior and tool topology, while fx and Grok use native models rather than the matched Luna-max family. Grok task repetition was censored by its free usage quota, and fx timed out on all three heavyweight trials.

1 Question and Estimands

The motivating question was not simply which CLI feels fastest. It was which installed harness places the smallest local burden on a four-core Linux machine, and whether that answer changes for ordinary maintenance work, heavyweight multi-package work, and delegation. The study therefore uses two estimands.

The first estimand is fresh-process startup footprint. Each executable ran `-version` and `-help` twenty times in a seeded sequential order. No model request was made. The second is deployment footprint: the full local process tree created by an agent while it attempted a fixed coding task. The second estimand deliberately includes model response handling and tool use; it answers a practical deployment question rather than isolating binary code size from all other work.

2 Systems and Controls

The matched task arms explicitly requested GPT-5.6 Luna max wherever available: OpenCode used `openai/gpt-5.6-luna` with variant `max`; Codex used `gpt-5.6-luna` with `model_reasoning_effort=max`; Cursor used `gpt-5.6-luna-max`; and Pi used `openai-codex/gpt-5.6-luna` with `-thinking max`. Grok Build used native Grok 4.6. fx used its native default `zai/glm-5.2`. The latter two are explicitly separate model exceptions rather than hidden claims of model equivalence.

The host was Linux x86-64 with four logical CPUs and approximately 15.5 GiB of RAM. Runs were sequential. Each task received a fresh fixture copy. The light fixture was a small retry-with-backoff maintenance issue with eight hidden checks. The heavy fixture contained three independent implementation, security, and comprehension packages with 24 hidden checks. All agent work was scored after the resource measurement window.

Solo arms disabled delegation where possible. Delegated arms enabled the harness-specific mechanism: bounded OpenCode workers, Codex multi-agent mode, Cursor task tools, and a Pi Luna-worker extension. fx exposed no comparable subagent control in this version, so it has a solo arm only.

3 Instrumentation

The runner samples Linux `/proc` every 20 ms and follows descendants of the launched CLI process. It records summed tree RSS, maximum individual RSS, CPU ticks, process count, thread count, context switches, and process I/O. Very short native binaries can exit before a sampling interval; startup commands therefore also run under GNU `time -v`, with child rusage supplying the fallback CPU and maximum-RSS values. This fallback is applied consistently to the startup phase.

The primary summaries are medians and interquartile ranges. Task cells contain only three trials, so no significance test or universal rank is claimed. A provider quota failure is classified as provider-blocked. A task timeout remains an observed timeout and is not silently deleted.

4 Cold Startup Results

Harness	Peak RSS	CPU seconds	Wall seconds
fx	1.47 MB	< 0.01	0.21
Grok Build	27.0 MB	0.02	0.22
Codex CLI	50.5 MB	0.09	0.24
Pi	157.9 MB	2.51	2.36
OpenCode	187.5 MB	1.85	1.67
Cursor Agent	206.4 MB	1.76	1.70

The result supports the strongest narrow conclusion of this study: fx is the lightest fresh native CLI in this environment. It does not establish that an active fx coding session remains the smallest after model context, tools, and long-running work are included.

5 Matched Luna-Max Task Results

Light maintenance, solo, three trials				
Harness	Peak RSS	CPU s	Wall s	Score
Pi	240.6 MB	3.70	52.6	8/8
Cursor	363.4 MB	6.23	59.1	8/8
Codex CLI	441.1 MB	4.79	72.7	8/8
OpenCode	742.8 MB	19.57	81.0	8/8

Heavy independent packages, solo, three trials				
Harness	Peak RSS	CPU s	Wall s	Median score
Pi	535.8 MB	18.86	1047.5	22/24
Cursor	565.0 MB	18.53	458.3	22/24
Codex CLI	608.9 MB	12.92	851.7	22/24
OpenCode	899.6 MB	49.52	836.5	22/24

The endpoints disagree by design. Pi is the lightest by peak tree RSS in both matched task types. Codex is the CPU-minimal heavy-task system. Cursor is the wall-time-minimal heavy-task system. The data do not support a single scalar claim that one harness is simply “lightest.”

6 Delegation

Delegation changed topology and usually increased heavy-task CPU. The median delegated-to-solo CPU ratios were 1.36 for OpenCode, 2.41 for Codex, 2.22 for Cursor, and 2.31 for Pi. Heavy-task wall ratios were 1.14, 1.17, 1.30, and 1.07 respectively. RSS ratios were not monotonic because child overlap and parent context release determine the peak.

This is not evidence that delegation is intrinsically wasteful. Delegation arms were asked to do more parallel work, and several scored 22–23/24 rather than a perfect 24/24. The correct interpretation is that the measured process tree price of the delegated deployment was generally higher in CPU, while the speed and memory consequences depended on the harness.

7 Native Exceptions

fx completed all three light tasks at 8/8. Its three heavy trials reached the 1800-second task limit with scores 2/24, 14/24, and 2/24. Its low local resource envelope is therefore not a heavy-task efficiency win. Grok Build completed one light solo task and one delegated heavy task before the free usage quota was exhausted; eleven later task attempts were provider-blocked. Grok is reported as a native-harness observation, not ranked against the repeated Luna arms.

8 Validity and Limitations

The design is valid for the narrow operational question of observed local resource envelopes under this host, these versions, these tasks, and these provider configurations. It is not a universal benchmark of harness quality or model efficiency. Limitations include one machine and service window, three task repetitions, cloud-provider behavior, a synthetic separable heavy fixture, no GPU or energy measurement, no persistent IDE-session measurement, unequal delegation implementations, and model exceptions for fx and Grok. The fixed process-tree instrumentation also cannot observe provider-side compute.

The task pair is inspired by common coding-agent evaluation shapes but is not SWE-bench, Terminal-Bench, or another official benchmark. The report therefore uses descriptive medians and explicit availability dispositions instead of pretending to supply a general industry leaderboard.

9 Conclusion

If the question is cold-start footprint, fx is the clear local minimum in this sample. If the question is a repeated GPT-5.6 Luna-max coding deployment, Pi used the least peak process-tree memory, Codex used the least CPU on the heavy task, and Cursor completed that heavy task fastest. OpenCode was the heaviest local tree in this particular configuration. The appropriate engineering choice depends on whether memory, CPU, latency, task completion, or delegation capacity is the binding constraint.